

Utilizing a Graph Neural Network (GNN) with Deep Q-Learning to Solve Dots and Boxes

Kevin Chen (Computer Systems - Yilmaz)
5/20/26

Background

- Dots and Boxes
 - $n \times m$ grid
 - $2nm + n + m$ edges
 - Each player takes turns drawing a line from one node to an adjacent node (no diagonal lines)
 - The player that creates a box gets one point And continues their turn until they can't create any more boxes
 - The player with the most boxes wins
 - Deterministic
 - No chance element
 - Perfect information

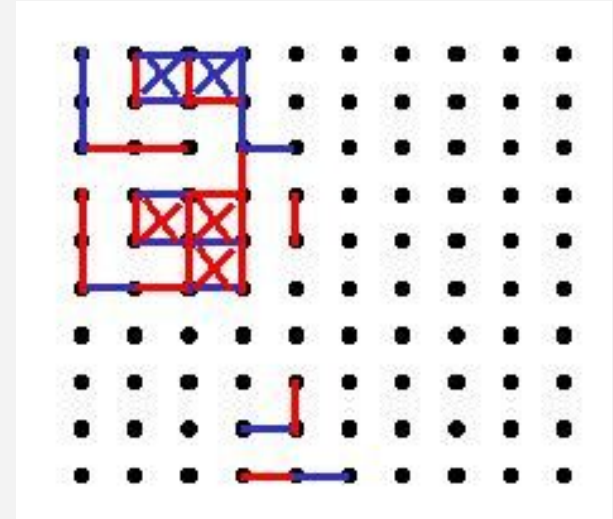


Figure 1: A game of Dots and Boxes

Background

- Terms^[1]
 - **Dot** - A point on the grid
 - **Line** - A connection between two dots
 - **Grid** - The playing zone
 - **Valence** - How many empty lines a box has
 - **Box** - The area enclosed by four lines
 - **Chain** - A sequence of boxes of valence two where two boxes share an empty line
 - **Loop** - A chain where the outermost empty line is shared

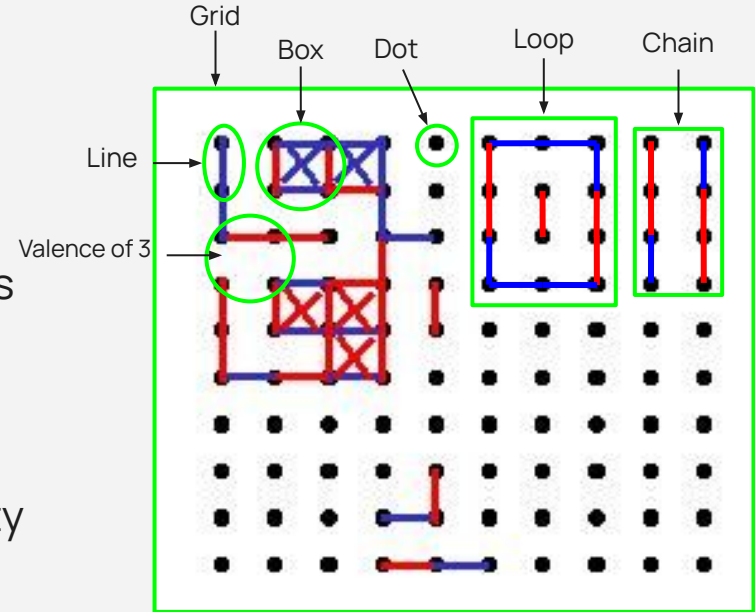


Figure 1: A labeled game of Dots and Boxes

Graph Neural Networks (GNNs)

- **Neural networks** are computational models inspired by the human brain that process, classify and analyze data to find patterns
- **Graph neural networks** are neural networks are designed to take in graphs as inputs
- GNNs learn through the process of **message passing**. They gather the information of their neighbors and aggregate it with their own information over several passes
- The number of passes represents the number of “layers” in the network.
- There are several variants of GNNs such as GCNs (Graph Convolutional Networks) and GATs (Graph Attention Networks) of of which change the method they aggregate and update the nodes in the graph.

Deep Q-Learning

- **Q-Learning** is a model-free, value-based reinforcement learning algorithm that allows models to learn based on interacting with their environment
 - Maps state-action pairs to expected future rewards using some function to maximize long term rewards (q-values)
 - These q-values are stored in a q-table
 - Explore by choosing a random move or exploit by choosing the highest q-value
- **Deep Q-Learning** combines Q-learning with neural networks
- As the environment becomes **larger**, the number of q-values to calculate **increases** significantly, so a **neural network** becomes necessary to estimate q-values more efficiently

Problem

- There are $2^{2mn + n + m}$ states where **n** is the length and **m** is the width of the board
 - State space grows exponentially (for example from a 1x1 to a 2x2, the state space grows 2^8 times bigger)
- There are no structural limitations to moves
 - You can play anywhere that has not been taken by the other player
 - Generates lots of state-action pairs



Existing Solutions

Rule-Based^[1]

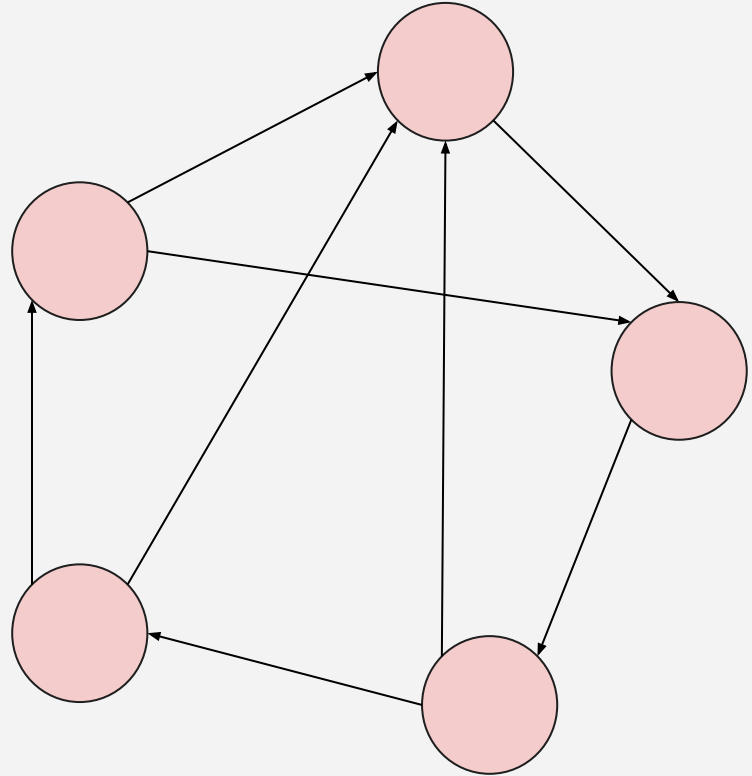
- Phase 1 (all boxes have valence > 1)
 - Be the first player to enter the second phase
 - Do not create boxes with valence 1
- Phase 2 (all boxes have valence ≤ 1)
 - Order chains and complete them from smallest to largest
 - Apply double dealing if beneficial
- Limitations: A greedy algorithm sacrifices very little and usually leads to big losses at the end

Alpha Beta Pruning^[1]

- A variation of the MiniMax algorithm
 - Reduces the number of nodes being searched
 - Assumes the opponent makes the best possible move
 - Minimize the maximum loss
 - Limitations
 - Still very expensive for large trees
 - Works best when best moves are evaluated first
 - Requires strong heuristics

Proposed Solution

- Dots and Boxes can be represented as a graph
- Utilizing reinforcement learning so a model can learn how to play the game optimally
- GNN as compared to a CNN
- Lightweight model



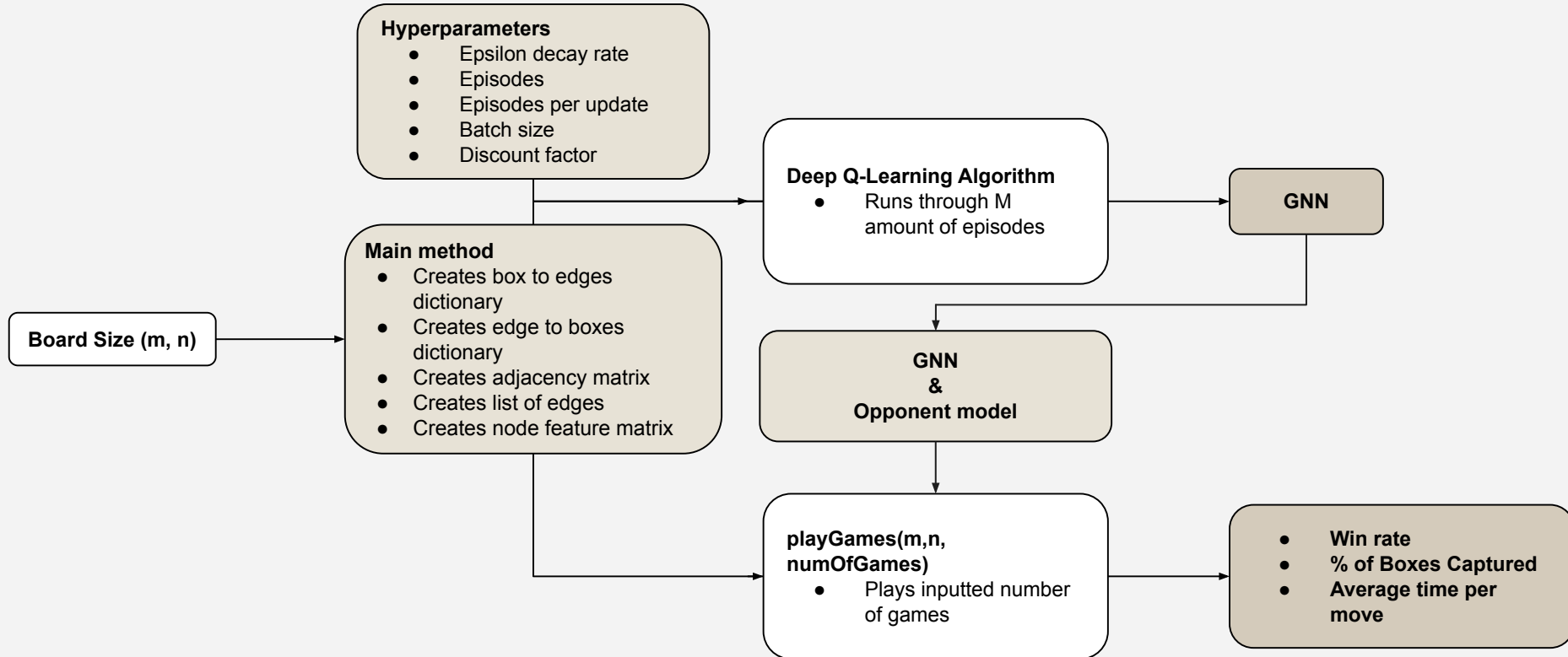
Novelty + Impact

- Leverages graph structure for improved generalization
- Makes computation for q-values more efficient than normal Q-Learning
- Generalizes gameplay across different board sizes
- Permutation invariant (recognizes game state symmetries)
- Viable method for other deterministic games that can be represented as graphs

The background features several overlapping circles in muted colors: light pink, beige, and light grey. A thin, wavy red line curves across the upper portion of the image.

Methodology

Program Outline



Board Representation

- Board (represented as a graph)
 - Each possible move (line) is a **node**
 - Dots are constant in every state, so a game state can be represented by edges alone
 - **Edges** connect moves that share a box
 - Connections will be represented in a **A**
 - The game state will be represented in a **node feature matrix**

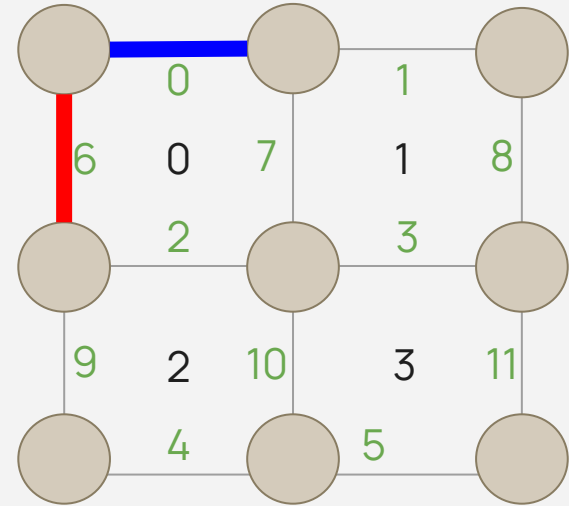


Figure 2: *Board-index representation*

Model Architecture

- 2-layer spatial graph convolutional network with Keras^[5]
- Turns the node feature matrix into a 64 dimension hidden vector and performs message passing two times
 - Two layers for the potential to learn the double dealing strategy, which requires to see two moves into the future

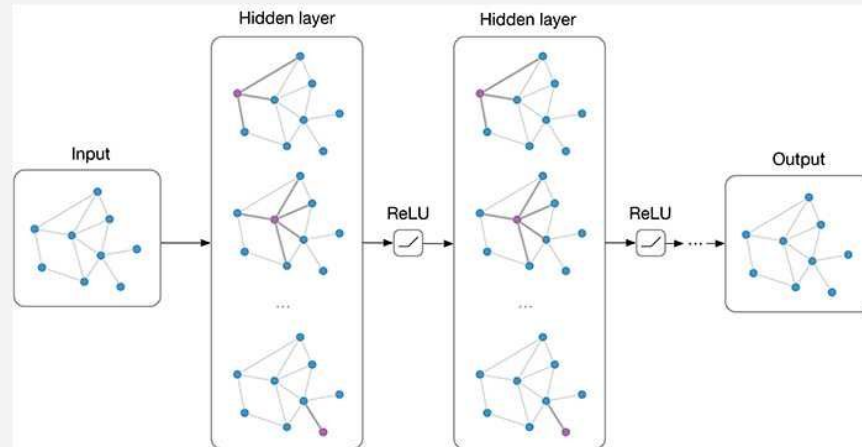


Figure 2: GNN Diagram^[6]

Message Passing Implementation

- Calculation: $H = \sigma(A \times N \times W)$
 - σ is the **activation function** (ReLU)
 - A is the **adjacency matrix** (connections between edges)
 - N is the **node feature matrix** (state of the board)
 - W is the **weight**
- Creates a sum-based aggregation
- H is used for intermediary calculations as N and reassigned
- The last H matrix is multiplied with another W_{out} in order to transform it into a vector (matrix with one column) where it maps each move to a q-value ($Q(s,a)$)

Adjacency Matrix

Line #	0	1	2	3	4	5	6	7	8	9	10	11
0	1	0	1	0	0	0	1	1	0	0	0	0
1	0	1	0	1	0	0	0	1	1	0	0	0
2	1	0	1	0	1	0	1	1	0	1	1	0
3	0	1	0	1	0	1	0	1	1	0	1	1
4	0	0	1	0	1	0	0	0	0	1	1	0
5	0	0	0	1	0	1	0	0	0	0	1	1
6	1	0	1	0	0	0	1	1	0	0	0	0
7	1	1	1	1	0	0	1	1	1	0	0	0
8	0	1	0	1	0	0	0	1	1	0	0	0
9	0	0	1	0	1	0	0	0	0	1	1	0
10	0	0	1	1	1	1	0	0	0	1	1	1
11	0	0	0	1	0	1	0	0	0	0	1	1

Figure 3: Representation of a 2x2 board in an adjacency matrix where the rows and columns represent the index of the lines in the board

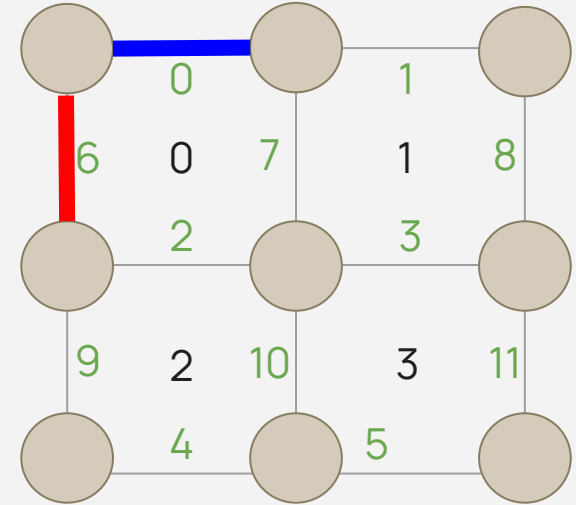


Figure 2: Board representation

Node Feature Matrix

Index	IsDrawn	IsOuter	Index	IsDrawn	IsOuter
0	1	1	6	1	1
1	0	1	7	0	0
2	0	0	8	0	1
3	0	0	9	0	1
4	0	1	10	0	0
5	0	1	11	0	1

Figure 4: Representation of a 2x2 board in a node feature matrix

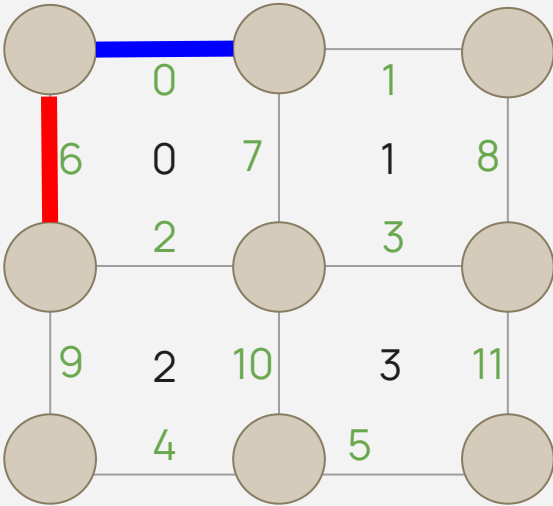


Figure 2: Board representation

Worked Example

A

1	0	1	0	0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	1	1	0	0	0
1	0	1	0	1	0	1	1	0	1	1	0
0	1	0	1	0	1	0	1	1	0	1	1
0	0	1	0	1	0	0	0	0	1	1	0
0	0	0	1	0	1	0	0	0	0	1	1
1	0	1	0	0	0	1	1	0	0	0	0
1	1	1	1	0	0	1	1	1	0	0	0
0	1	0	1	0	0	0	1	1	0	0	0
0	0	1	0	1	0	0	0	0	1	1	0
0	0	1	1	1	1	0	0	0	1	1	1
0	0	0	1	0	1	0	0	0	0	1	1

N

1	1
0	1
0	0
0	0
0	1
0	1
1	1
0	0
0	1
0	1
0	1
0	1



AN

2	2
0	2
2	5
0	5
0	3
0	3
2	2
2	4
0	2
0	3
0	5
0	5

Figure 5: Example of matrix multiplication and the resulting **sum-based aggregation**

Deep Q-Learning Algorithm

Algorithm 1 Deep Q-Learning Algorithm^[3]

```
1: Initialize replay memory  $D$  to capacity  $N$ 
2: Initialize action-value function  $Q$  with weights  $\theta$ 
3: Initialize target action-value function  $\hat{Q}$  with weights  $\theta^- = \theta$ 
4: for episode = 1,  $M$  do
5:   Initialize initial game state  $s_1 = \{x_1\}$ 
6:   for  $t = 1, T$  do
7:     With probability  $\varepsilon$  select a random action  $a_t$ 
8:     otherwise select  $a_t = \operatorname{argmax}_a Q(\phi(s_t, a; \theta))$ 
9:     Execute action  $a_t$  in emulator and observe reward
10:    Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
11:    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $D$ 
12:    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $D$ 
13:
14:    Set  $y_j = \begin{cases} r_j & \text{if episode terminates at step } j + 1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$ 
15:    Perform a gradient decent step on  $(y_j - Q(\phi_{j+1}, a; \theta^-))^2$  with respect to the
    network parameters  $\theta$ 
16:   end for
17:   Every  $C$  episodes reset  $\hat{Q} = Q$ 
18: end for
```

Figure 6: *The deep q-learning algorithm*^[4]

Reward Calculation

+1	For every box made
-1	For every box enemy makes
-10	For illegal moves (just in case)

Experimentation

Comparing win rate, boxes captured, and average time per move to these models on 100 games:

- Random
- Alpha beta^[4]
- Heuristics

On these size boards:

- 3x3, 4x4, 5x5

Statistics

- Win rate
 - $(\text{number of games player won} / \text{number of total games}) * 100$
- Boxes Captured
 - $(\text{Number of boxes for player } x / \text{total number of boxes for both players}) * 100$
- Average time per move
 - $(\sum \text{time per move from player } x) / (\# \text{ of moves from player } x)$

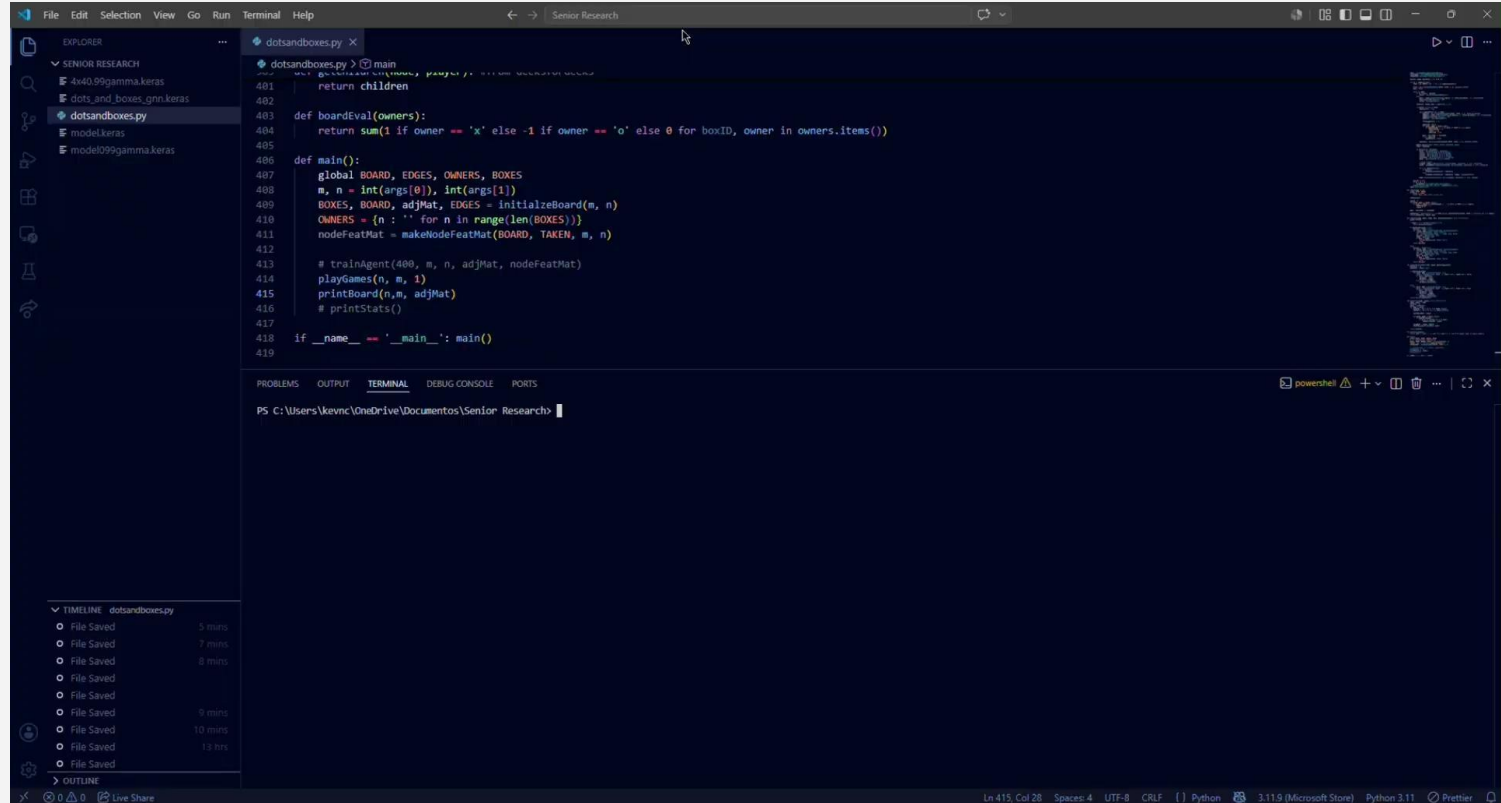
Training Process

- Trained on 4x4 and 400 episodes
- Hyperparameters
 - Epsilon Decay Factor: 0.99
 - Discount Factor: 0.99
 - Updates Target Network after every 10 episodes
 - Batch size: 32

Results

Opponent	Win Rate	Boxes Captured	Time per move for GNN (s)	Time per move for opponent (s)
Alpha Beta of depth 3 (3x3)	71%	60.7%	0.0563	0.0218
Alpha Beta of depth 3 (4x4)	86%	72.56%	0.0544	0.0925
Alpha Beta of depth 3 (4x4)	88%	73.20%	0.0569	6.866
Random (3x3)	72%	59.22%	0.0567	5.62e-6
Random (4x4)	83%	72.31%	0.0785	1.406e-5
Random (5x5)	85%	70.84%	0.0995	8.454e-6
Heuristics (3x3)	71%	59.56%	0.0544	2.177e-5
Heuristics (4x4)	83%	70.69%	0.0538	3.767e-5
Heuristics (5x5)	94%	77.52%	0.0531	5.167e-5

Demo Video



Conclusion

- Goal: develop a model that achieved a win rate of 80% against other models
- Achieved a win rate average of 81% across all experiments
- Average of 68.51% boxes captured
- Achieved a superior time complexity, where increasing the board size had a negligible effect on the time to compute a move
- Generalization across board sizes was a little lacking

Limitations

- Training the model for 400 episodes took about 40 on a 4x4 board
 - Most likely due to the nature of matrix multiplication and the nature of the Python environment
- The model's performance is inversely proportional to the size of the board compared to what it was trained on
 - It was trained on bigger boards, so it might oversmooth the moves on smaller boards, where on larger boards, it can apply strategies locally

Future Work

- Test my model against a larger variety of models
- Switch algorithms as search size decreases
- More attributes in the node feature matrix
- Include a time limit for choosing a move in gameplay
- Utilize Graph Attention Networks (GATs) to put weights on certain moves
 - Ranks loops and chains

References

- [1] E. Simon, "ARTIFICIAL, INTELLIGENT AGENTS FOR DOTS AND BOXES," PlayItNice, 2021.
https://www.academia.edu/49067480/ARTIFICIAL_INTELLIGENT_AGENTS_FOR_DOTS_AND_BOXES (accessed Apr. 23, 2026).
- [2] A. Pandey, A. Charles, J. Ross, and C. Edwin, "Solving dots & boxes using reinforcement learning," Mountainscholar.org, 2022. <https://mountainscholar.org/items/678cab99-8205-46f9-b664-5281ebd6db18> (accessed Apr. 23, 2026).
- [3] "The Deep Q-Learning Algorithm - Hugging Face Deep RL Course," huggingface.co.
<https://huggingface.co/learn/deep-rl-course/unit3/deep-q-algorithm>
- [4] GeeksforGeeks, "AlphaBeta pruning in Adversarial Search Algorithms," GeeksforGeeks, Jun. 13, 2024.
<https://www.geeksforgeeks.org/artificial-intelligence/alpha-beta-pruning-in-adversarial-search-algorithms/>
- [5] K. Team, "Keras documentation: Node Classification with Graph Neural Networks," keras.io.
https://keras.io/examples/graph/gnn/_citations/
- [6] S. Karagiannakos, "Graph Neural Networks - An overview," AI Summer, Feb. 01, 2020.
https://theaisummer.com/Graph_Neural_Networks/

Questions?

Thank You!